Problem 1 [20 points] CNN

Consider the following 4 × 4 image and 2 × 2 filter below.

1	3	-2	4
0	8	6	5
2	1	-9	0
4	-1	3	7

1	2
-2	-1

1 [6 points]. Assume that there is no padding and stride = 1. What are the dimensions of the output, and what is the value in the bottom right corner of the output image?

Solution (1.)

Dimensions of the output = **3x3**

The bottom right corner of the output image is -22.

$$\begin{bmatrix} -9 & 0 \\ 3 & 7 \end{bmatrix} * \begin{bmatrix} 1 & 2 \\ -2 & -1 \end{bmatrix} = (-9 * 1) + (0 * 2) + (3 * -2) + (7 * -1) = -22$$

2 [6 points]. Now assume that we have padding = 1. Given that, what are the new dimensions of the output, and the new value in the bottom right corner?

Solution (2.)

With the padding = 1, the new image dimensions is 6x6 while the filter size doesn't change.

0	0	0	0	0	0
0	1	3	-2	4	0
0	0	8	6	5	0
0	2	1	-9	0	0
0	4	-1	3	7	0
0	0	0	0	0	0

1	2
-2	-1

The dimensions of the output = 5 x 5

The new value in the bottom right corner is 7.

The new value in the bottom right corner is.

$$\begin{bmatrix} 7 & 0 \\ 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 2 \\ -2 & -1 \end{bmatrix} = (7*1) + (0*2) + (0*-2) + (0*-1) = 7$$

3 [8 points]. Suppose we have below 2-channel input I and filter F as follows. What is the dimensions of the output, assume padding=0 and stride=1, and what is the value in the bottom right corner of the output image?

$$I[0,:,:] = \begin{pmatrix} 4 & 2 & 1 & 3 \\ 6 & 1 & -2 & -3 \\ -1 & 3 & 4 & 0 \end{pmatrix}$$

$$F[0,:,:] = \begin{pmatrix} 1 & -2 \\ 0 & 1 \end{pmatrix}$$

$$I[1,:,:] = \begin{pmatrix} -2 & 1 & 4 & 5 \\ 0 & 6 & 3 & 6 \\ 4 & -1 & 1 & 1 \end{pmatrix}$$

$$F[1,:,:] = \begin{pmatrix} 1 & 0 \\ 1 & -1 \end{pmatrix}$$

Solution (3.)

The dimensions of the output = 2 x 3

For the input zero $I[0,:,:]$ and filter zero $F[0,:,:]$.

We have.

$$\begin{pmatrix} 4 & 2 & 1 & 3 \\ 6 & 1 & -2 & -3 \\ -1 & 3 & 4 & 0 \end{pmatrix} * \begin{pmatrix} 1 & -2 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & -2 & -8 \\ 7 & 9 & \mathbf{4} \end{pmatrix}$$

Bottom right corner = $\begin{pmatrix} -2 & -3 \\ 4 & 0 \end{pmatrix} * \begin{pmatrix} 1 & -2 \\ 0 & 1 \end{pmatrix} = (-2*1) + (-3*-2) + (4*0) + (0*1) = 4$

$$= -2 + 6 + 0 + 0 = 4$$

For the input one $I[1,:,:]$ and filter one $F[1,:,:]$.

We have

$$\begin{pmatrix} -2 & 1 & 4 & 5 \\ 0 & 6 & 3 & 6 \\ 4 & -1 & 1 & 1 \end{pmatrix} * \begin{pmatrix} 1 & 0 \\ 1 & -1 \end{pmatrix} = \begin{pmatrix} -8 & 4 & 1 \\ 5 & 4 & \mathbf{3} \end{pmatrix}$$

The bottom right corner = $\begin{pmatrix}3 & 6\\ 1 & 1\end{pmatrix}\begin{pmatrix}1 & 0\\ 1 & -1\end{pmatrix} = (3*1)+(6*0)+(1*1)+(1*-1)$

$$= 3+0+1-1=3$$

The output of the two channels is.

$$\begin{pmatrix}1 & -2 & -8\\ 7 & 9 & \mathbf{4}\end{pmatrix}+\begin{pmatrix}-8 & 4 & 1\\ 5 & 4 & \mathbf{3}\end{pmatrix}=\begin{pmatrix}-7 & 2 & -7\\ 12 & 13 & \mathbf{7}\end{pmatrix}$$

Therefore, the bottom right corner of the output image = $4+3=7$

Problem 2 [25 points] Recurrent Neural Network

Consider the following 1D RNN with no nonlinearities, a 1D hidden state, and 1D inputs u_t at each timestep. (Note: There is only a single parameter w, no bias). This RNN expresses unrolling the following recurrence relation, with hidden state h_t at unrolling step t given by:

$$h_t = w.(u_t + h_{t-1})$$

The computational graph of unrolling the RNN for three timesteps is shown below, where w is the learnable weight, u_1, u_2, and u_3 are sequential inputs, and p, q, r, s,and t are intermediate values.

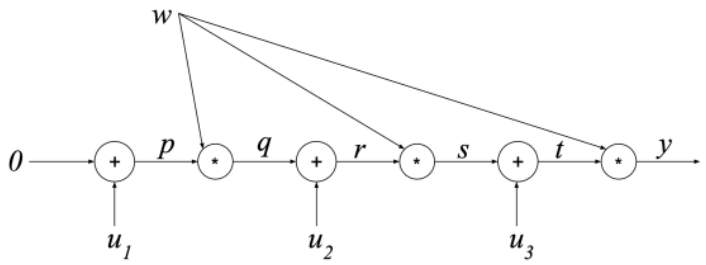

Figure 1: Computational graph in RNN for three sequential inputs

1 [8 points]. Given parameter w, and inputs u_1, u_2, u_3, please write out the expression of $p, q, r, s, t,$ and output y.

Solution (1.)

$$p = u_1 + 0 = u_1$$

$$q = w.p = w.u_1$$

$$r = u_2 + q = u_2 + w.u_1$$

$$s = w.r = w.(u_2 + w.u_1) = w.u_2 + w^2.u_1$$

$$t = u_3 + s = u_3 + w.u_2 + w^2.u_1$$

$$y = w.t = w.(u_3 + w.u_2 + w^2.u_1) = w.u_3 + w^2.u_2 + w^3 u_1$$

2 [8 points]. Why does RNN have memory and how does it achieve this function? Comparing RNN against LSTM, what is the drawback?

Solution (2.)

Recurrent Neural Networks (RNNs) possess memory due to their ability to maintain a hidden state that retains information from past inputs through recurrent connections. This hidden state serves as a form of memory, allowing RNNs to process sequential data by incorporating contextual information from previous time steps. However, a significant drawback of traditional RNNs is the vanishing gradient problem, where gradients diminish exponentially as they propagate backward through time, making it challenging for the network to learn long-term dependencies effectively. In contrast, Long Short-Term Memory (LSTM) networks address this limitation by incorporating specialized gating mechanisms that regulate the flow of information, enabling them to capture and retain long-range dependencies more effectively than traditional RNNs.

3 [9 points]. Given the LSTM architecture as shown in Figure 2. Suppose you want the memory cell to sum its inputs over time. What values should the input gate and forget gate take?

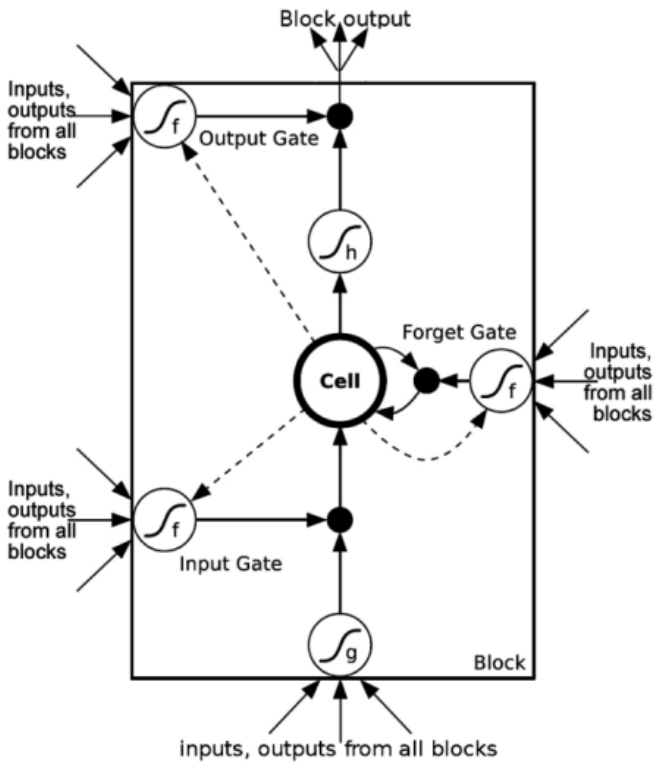

Figure 2: LSTM architecture

Solution (3.)

To achieve the goal of summing inputs over time:

The input gate should be fully open, allowing all information from the current time step to be written to the memory cell. So, the input gate should be set to 1.

The forget gate should also be fully open, allowing the retention of all information from the previous time steps. So, the forget gate should be set to 1.

In summary, for the memory cell to sum its inputs over time, both the input gate and forget gate should take the value of 1.

Problem 3 [30 points] Query-Key-Value Mechanics in Self-Attention

Self-attention is the core building block for the Transformer model, which has kickstarted the amazing progress in seq2seq modeling. In this question, you will be studying the detailed mechanics of the Query-Key-Value interaction in a self-attention block in processing a sequence. Here we will be studying the case where the encoder only takes in 3 inputs, with the variables defined as:

$$a_1 = \begin{bmatrix}5\\1\end{bmatrix}\ a_2 = \begin{bmatrix}4\\3\end{bmatrix}\ a_3 = \begin{bmatrix}-2\\6\end{bmatrix}\ W_q = \begin{bmatrix}1 & 0\\2 & 9\end{bmatrix}\ W_k = \begin{bmatrix}0 & 1\\6 & 0\end{bmatrix}\ W_v = \begin{bmatrix}4 & 3\\9 & 1\end{bmatrix}$$

where h_i denotes the hidden states at timestep i at a arbitrary self-attention layer. Each q_i , k_i, $v_i$is determined by $q_i = W_q a_i$, $k_i = W_k a_i$, $v_i = W_v a_i$.

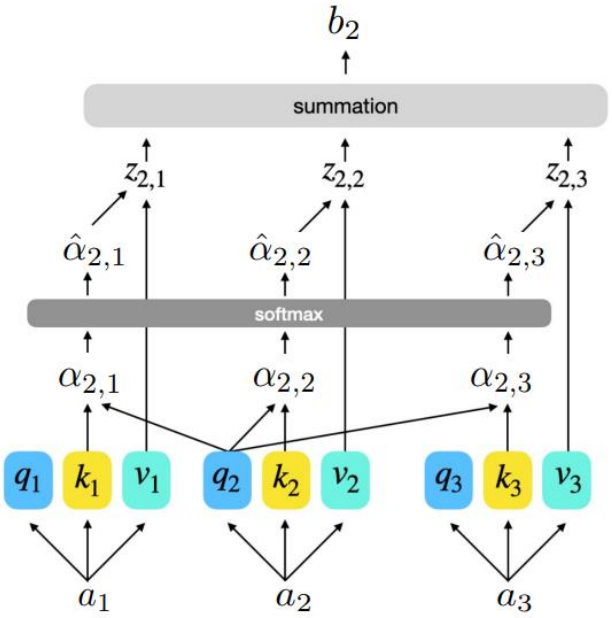

Figure 3: Self-attention of timestep 2 in encoder

1 [10 points]. What are $\propto_{2,1}$, $\propto_{2,2}$, $\propto_{2,3}$,? Compute their values.

Solution (1.)

$$\propto_{2,i} = q_2^T * k_i/\sqrt{d}$$

Since,

W_q has dimensions of 2×2, implying a hidden state dimension of 2.

W_k has dimensions of 2×2, implying a hidden state dimension of 2.

W_v has dimensions of 2×2, implying a hidden state dimension of 2.

Therefore, the dimensionality d is 2.

$$\propto_{2,1} = q_2^T * k_1$$

$$\propto_{2,2} = q_2^T * k_2$$

$$\propto_{2,3} = q_2^T * k_3$$

$$q_2 = W_q a_2 = \begin{bmatrix}1 & 0\\ 2 & 9\end{bmatrix}\begin{bmatrix}4\\3\end{bmatrix} = \begin{bmatrix}(1*4)+(0*3)\\(2*4)+(9*3)\end{bmatrix} = \begin{bmatrix}4+0\\8+27\end{bmatrix} = \begin{bmatrix}4\\35\end{bmatrix}$$

$$k_1 = W_k a_1 = \begin{bmatrix}0 & 1\\ 6 & 0\end{bmatrix}\begin{bmatrix}5\\1\end{bmatrix} = \begin{bmatrix}(0*5)+(1*1)\\(6*5)+(0*1)\end{bmatrix} = \begin{bmatrix}0+1\\30+0\end{bmatrix} = \begin{bmatrix}1\\30\end{bmatrix}$$

Therefore, our $\propto_{2,1}$ becomes

$$\propto_{2,1} = q_2^T * k_1/\sqrt{d} = [4 \quad 35]\begin{bmatrix}1\\30\end{bmatrix} = [(4*1)+(35*30)] = [4+1050] = 1054/\sqrt{2}$$

Similarly, $\propto_{2,2}$ is computed as follows.

$$\propto_{2,2} = q_2^T * k_2$$

$$k_2 = W_k a_2 = \begin{bmatrix}0 & 1\\ 6 & 0\end{bmatrix}\begin{bmatrix}4\\3\end{bmatrix} = \begin{bmatrix}(0*4)+(1*3)\\(6*4)+(0*3)\end{bmatrix} = \begin{bmatrix}0+3\\24+0\end{bmatrix} = \begin{bmatrix}3\\24\end{bmatrix}$$

Now,

$$\propto_{2,2} = q_2^T * k_2/\sqrt{d} = [4 \quad 35]\begin{bmatrix}3\\24\end{bmatrix} = [(4*3)+(35*24)] = [12+840] = 852/\sqrt{2}$$

Similarly, $\propto_{2,3}$ is computed in the same way.

$$\propto_{2,3} = q_2^T * k_3$$

$$k_3 = W_k a_3 = \begin{bmatrix}0 & 1\\ 6 & 0\end{bmatrix}\begin{bmatrix}-2\\6\end{bmatrix} = \begin{bmatrix}(0*-2)+(1*6)\\(6*-2)+(0*6)\end{bmatrix} = \begin{bmatrix}0+6\\-12+0\end{bmatrix} = \begin{bmatrix}6\\-12\end{bmatrix}$$

Now,

$$\propto_{2,3} = q_2^T * k_3/\sqrt{d} = [4 \quad 35]\begin{bmatrix}6\\-12\end{bmatrix} = [(4*6)+(35*-12)] = [24-420] = -396/\sqrt{2}$$

2 [10 points]. Figure 3 shows the computational graph of the three inputs in a transformer layer. Write out the operation process from input a_1, a_2, a_3 to output b_2. You don't need to calculate the values but have to show the equation in each steps.

Solution (2.)

To write the operation from input a_1, a_2, a_3 to output b_2. The output b_2 is computed as follows.

$$q_i = W_q\, a_i$$

$$k_i = W_k\, a_i$$

$$v_i = W_v\, a_i$$

$$\alpha_{2,i} = q_2 * k_i/\sqrt{d}$$

$$\alpha_{2,1} = q_2 * k_1/\sqrt{d}$$

$$\alpha_{2,2} = q_2 * k_2/\sqrt{d}$$

$$\alpha_{2,3} = q_2 * k_3/\sqrt{d}$$

$$b_2 = \sum_i \hat{\alpha}_{2,i}\, v_i$$

$$b_2 = \hat{\alpha}_{2,1} . v_1 + \hat{\alpha}_{2,2}. v_2 + \hat{\alpha}_{2,3}. v_3$$

$$\hat{\alpha}_{2,i} = \frac{\exp(\alpha_{2,i})}{\sum_j \exp(\alpha_{2,j})}$$

$$\hat{\alpha}_{2,1} = \frac{\exp(\alpha_{2,1})}{\sum_j \exp(\alpha_{2,j})}$$

$$\hat{\alpha}_{2,2} = \frac{\exp(\alpha_{2,2})}{\sum_j \exp(\alpha_{2,j})}$$

$$\hat{\alpha}_{2,3} = \frac{\exp(\alpha_{2,3})}{\sum_j \exp(\alpha_{2,j})}$$

$$Recall: v_i = W_v\, a_i$$

$$Z_2, i = \hat{\alpha}_{2,i} . v_i$$

By substituting these values into the output b_2, we then have.

$$b_2 = \sum_i Z_{2,i}$$

$$b_2 = Z_{2,1} + Z_{2,2} + Z_{2,3}$$

$$b_2 = \hat{\alpha}_{2,1} . v_1 + \hat{\alpha}_{2,2} . v_2 + \hat{\alpha}_{2,3} . v_3$$

3 [10 points]. In practice, it is common to have multiple self-attention operations happening in one layer. Each individual self-attention component is referred as a head, and thus the entire block is typically called Multi-head Self-Attention. What's the benefit of having multiple heads? (Hint: why do we want multiple kernel filters in CNN?)

Solution (3.)

Having multiple heads in a Multi-head Self-Attention mechanism provides several benefits which include.

- Increased Representation Capacity: Each head can learn different relationships between words or tokens in the input sequence. By having multiple heads, the model can capture different aspects of the input data simultaneously. This leads to a richer representation of the input, allowing the model to learn more complex patterns and dependencies.
- Enhanced Robustness and Generalization: Different heads may attend to different parts of the input sequence, allowing the model to focus on both local and global dependencies. This enhances the model's ability to generalize across various tasks and input types. Additionally, it makes the model more robust to noise or irrelevant information in the input.
- Reduced Overfitting: By learning multiple attention distributions, the model effectively regularizes itself. Each head focuses on different aspects of the input, which reduces the risk of overfitting to specific patterns in the data. This leads to improved generalization performance, especially in scenarios with limited training data.
- Parallelization: Each attention head operates independently, allowing for parallel computation during training and inference. This can significantly speed up the training process and make the model more efficient, especially on hardware with parallel processing capabilities like GPUs or TPUs.
- Interpretability: In some cases, each attention head can be interpreted as learning different linguistic or semantic relationships within the input sequence. This can provide insights into how the model makes predictions and aid in model debugging or analysis.

Overall, the use of multiple heads in Multi-head Self-Attention enables the model to learn richer representations, improve generalization performance, reduce overfitting, facilitate parallelization, and potentially offer interpretability benefits.

Problem 4 [25 points] Graph Neural Network

Consider the following undirected graph G: In this problem, we are going to work with an undirected

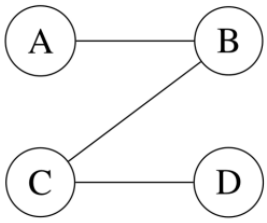

Figure 4: Graph for GNN forward pass

graph without edge weights. We will walk through a simplified forward pass of graph neural networks to help build concrete intuition for how GNNs operate. To begin with, let us assign vectors v_A, v_B, v_C, v_D, to nodes A, B, C, D:

$$v_A = \begin{bmatrix}1\\2\end{bmatrix}\ v_B = \begin{bmatrix}-1\\2\end{bmatrix}\ v_C = \begin{bmatrix}2\\2\end{bmatrix}\ v_D = \begin{bmatrix}-3\\2\end{bmatrix}$$

For each layer of GCN, we have the computation as

$$h_v^k = \sigma\left(W_k \sum_{u \in N(v) \cup v} \frac{h_u^{k-1}}{\sqrt{|N(u)||N(v)|}} \right),$$

where the activation function $\sigma(.)$is $ReLU(.) = \max(0, .)$. W_1 for the 1st GNN layer, and W_2 for the 2nd GNN layer are

$$W_1 = \begin{bmatrix} 3 & 3 \\ 2 & 1 \end{bmatrix} W_2 = \begin{bmatrix} 2 & 5 \\ 4 & 2 \end{bmatrix}$$

1 [5 points]. What is the adjacency matrix of the graph?

Solution (1.)

The adjacency matrix is used to store the graph in a computer memory. For the given graph the adjacency matrix dimensions is 4 x 4 because the nodes are four in number. The row and the column represent the nodes of the graph. If edge is present, then I store 1 otherwise I store 0. With this I created the adjacency matrix of the graph as shown below.

	A	B	C	D
A	0	1	0	0
B	1	0	1	0
C	0	1	0	1
D	0	0	1	0

2 [8 points]. What is the first GCN layer output h_v^1 for node A, B, C, D? (h_v^0 is the node features v).

Solution (2.)

For the first GCN layer output h_v^1.

To calculate h_v^1 for nodes A, B, C, D, I apply the GCN layer equation with W_1 and $\sigma = ReLU$:

For Node A.

$$h_A^1 = ReLU\left(W_1 * \frac{v_A}{\sqrt{N(A)||N(v_A)}} + \frac{v_B}{\sqrt{N(B)||N(v_A)}} \right)$$

$$h_A^1 = ReLU\left(\begin{bmatrix} 3 & 3 \\ 2 & 1 \end{bmatrix} * \left(\frac{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}{\sqrt{(1*1)}} + \frac{\begin{bmatrix} -1 \\ 2 \end{bmatrix}}{\sqrt{(2*1)}} \right) \right)$$

$$h_A^1 = ReLU\left(\begin{bmatrix} 3 & 3 \\ 2 & 1 \end{bmatrix} * \left(\frac{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}{1} + \frac{\begin{bmatrix} -1 \\ 2 \end{bmatrix}}{\sqrt{2}} \right) \right)$$

$$h_A^1 = ReLU\left(\left(\begin{bmatrix}9\\4\end{bmatrix} + \frac{1}{\sqrt{2}}\begin{bmatrix}3\\0\end{bmatrix}\right)\right)$$

$$h_A^1 = ReLU\left(\begin{bmatrix}9 + {}^{3}/\sqrt{2}\\4\end{bmatrix}\right)$$

$$h_A^1 = ReLU([9 + 2.121; 4])$$

$$h_A^1 = ReLU\left(\begin{bmatrix}11.121\\4\end{bmatrix}\right)$$

$$h_A^1 = \begin{bmatrix}11.121\\4\end{bmatrix}$$

For Node B.

$$h_B^1 = ReLU\left(W_1 * \frac{v_A}{\sqrt{N(A)||N(v_B)}} + \frac{v_B}{\sqrt{N(B)||N(v_B)}} + \frac{v_C}{\sqrt{N(C)||N(v_C)}}\right)$$

$$h_B^1 = ReLU\left(\begin{bmatrix}3 & 3\\2 & 1\end{bmatrix} * \frac{\begin{bmatrix}1\\2\end{bmatrix}}{\sqrt{1*2}} + \frac{\begin{bmatrix}-1\\2\end{bmatrix}}{\sqrt{2*2}} + \frac{\begin{bmatrix}2\\2\end{bmatrix}}{\sqrt{2*2}}\right)$$

$$h_B^1 = ReLU\left(\begin{bmatrix}3 & 3\\2 & 1\end{bmatrix} * \left(\frac{\begin{bmatrix}1\\2\end{bmatrix}}{\sqrt{2}} + \frac{\begin{bmatrix}-1\\2\end{bmatrix}}{\sqrt{4}} + \frac{\begin{bmatrix}2\\2\end{bmatrix}}{\sqrt{4}}\right)\right)$$

$$h_B^1 = ReLU\left(\begin{bmatrix}3 & 3\\2 & 1\end{bmatrix} * \left(\frac{\begin{bmatrix}1\\2\end{bmatrix}}{\sqrt{2}} + \frac{\begin{bmatrix}-1\\2\end{bmatrix}}{2} + \frac{\begin{bmatrix}2\\2\end{bmatrix}}{2}\right)\right)$$

$$h_B^1 = ReLU\left(\begin{bmatrix}3 & 3\\2 & 1\end{bmatrix} * \frac{1}{\sqrt{2}}\begin{bmatrix}1\\2\end{bmatrix} + \begin{bmatrix}3 & 3\\2 & 1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}-1\\2\end{bmatrix} + \begin{bmatrix}3 & 3\\2 & 1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}2\\2\end{bmatrix}\right)$$

$$h_B^1 = ReLU\left(\frac{1}{\sqrt{2}}\begin{bmatrix}3+6\\2+2\end{bmatrix} + \frac{1}{2}\begin{bmatrix}-3+6\\-2+2\end{bmatrix} + \frac{1}{2}\begin{bmatrix}6+6\\4+2\end{bmatrix}\right)$$

$$h_B^1 = ReLU\left(\frac{1}{\sqrt{2}}\begin{bmatrix}9\\4\end{bmatrix} + \frac{1}{2}\begin{bmatrix}3\\0\end{bmatrix} + \frac{1}{2}\begin{bmatrix}12\\6\end{bmatrix}\right)$$

$$h_B^1 = ReLU\left(\begin{bmatrix}6.364\\2.829\end{bmatrix} + \begin{bmatrix}1.5\\0\end{bmatrix} + \begin{bmatrix}6\\3\end{bmatrix}\right)$$

$$h_B^1 = ReLU\left(\begin{bmatrix}13.864\\5.829\end{bmatrix}\right)$$

$$h_B^1 = \begin{bmatrix}13.864\\5.829\end{bmatrix}$$

For Node C:

$$h_C^1 = ReLU\left(W_1 * \left(\frac{v_B}{\sqrt{N(B)||N(v_C)}} + \frac{v_C}{\sqrt{N(C)||N(v_C)}} + \frac{v_D}{\sqrt{N(D)||N(v_C)}}\right)\right)$$

$$h_C^1 = ReLU\left(\begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix} * \left(\frac{\begin{bmatrix}-1\\ 2\end{bmatrix}}{\sqrt{2*2}} + \frac{\begin{bmatrix}2\\ 2\end{bmatrix}}{\sqrt{2*2}} + \frac{\begin{bmatrix}-3\\ 2\end{bmatrix}}{\sqrt{1*2}}\right)\right)$$

$$h_C^1 = ReLU\left(\begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix} * \left(\frac{\begin{bmatrix}-1\\ 2\end{bmatrix}}{\sqrt{4}} + \frac{\begin{bmatrix}2\\ 2\end{bmatrix}}{\sqrt{4}} + \frac{\begin{bmatrix}-3\\ 2\end{bmatrix}}{\sqrt{2}}\right)\right)$$

$$h_C^1 = ReLU\left(\begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix} * \left(\frac{\begin{bmatrix}-1\\ 2\end{bmatrix}}{2} + \frac{\begin{bmatrix}2\\ 2\end{bmatrix}}{2} + \frac{\begin{bmatrix}-3\\ 2\end{bmatrix}}{\sqrt{2}}\right)\right)$$

$$h_C^1 = ReLU\left(\begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}-1\\ 2\end{bmatrix} + \begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix} * \frac{1}{2}\begin{bmatrix}2\\ 2\end{bmatrix} + \begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix} * \frac{1}{\sqrt{2}}\begin{bmatrix}-3\\ 2\end{bmatrix}\right)$$

$$h_C^1 = ReLU\left(\frac{1}{2}\begin{bmatrix}-3+6\\ -2+2\end{bmatrix} + \frac{1}{2}\begin{bmatrix}6+6\\ 4+2\end{bmatrix} + \frac{1}{\sqrt{2}}\begin{bmatrix}-9+6\\ -6+2\end{bmatrix}\right)$$

$$h_C^1 = ReLU\left(\frac{1}{2}\begin{bmatrix}3\\ 0\end{bmatrix} + \frac{1}{2}\begin{bmatrix}12\\ 6\end{bmatrix} + \frac{\sqrt{2}}{2}\begin{bmatrix}-3\\ -4\end{bmatrix}\right)$$

$$h_C^1 = ReLU\left(\begin{bmatrix}1.5\\ 0\end{bmatrix} + \begin{bmatrix}6\\ 3\end{bmatrix} + \begin{bmatrix}-2.121\\ -2.828\end{bmatrix}\right)$$

$$h_C^1 = ReLU\left(\begin{bmatrix}5.379\\ 0.172\end{bmatrix}\right)$$

$$h_C^1 = \begin{bmatrix}5.379\\ 0.172\end{bmatrix}$$

For Node D:

$$h_D^1 = ReLU\left(W_1 * \left(\frac{v_C}{\sqrt{N(C)||N(v_D)}} + \frac{v_D}{\sqrt{N(D)||N(v_D)}} +\right)\right)$$

$$h_D^1 = ReLU\left(\begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix} * \left(\frac{\begin{bmatrix}2\\ 2\end{bmatrix}}{\sqrt{(2*1)}} + \frac{\begin{bmatrix}-3\\ 2\end{bmatrix}}{\sqrt{(1*1)}}\right)\right)$$

$$h_D^1 = ReLU\left(\begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix} * \left(\frac{\begin{bmatrix}2\\ 2\end{bmatrix}}{\sqrt{2}} + \frac{\begin{bmatrix}-3\\ 2\end{bmatrix}}{1}\right)\right)$$

$$h_D^1 = ReLU\left(\begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix} * \left(\frac{1}{\sqrt{2}}\begin{bmatrix}2\\ 2\end{bmatrix} + \begin{bmatrix}-3\\ 2\end{bmatrix}\right)\right)$$

$$h_D^1 = ReLU\left(\begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix} * \left(\frac{\sqrt{2}}{2}\begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix}\begin{bmatrix}2\\ 2\end{bmatrix} + \begin{bmatrix}3 & 3\\ 2 & 1\end{bmatrix}\begin{bmatrix}-3\\ 2\end{bmatrix}\right)\right)$$

$$h_D^1 = ReLU\left(\frac{\sqrt{2}}{2}\begin{bmatrix}6+6\\ 4+2\end{bmatrix} + \begin{bmatrix}-9+6\\ -6+2\end{bmatrix}\right)$$

$$h_D^1 = ReLU\left(\frac{\sqrt{2}}{2}\begin{bmatrix}12\\ 6\end{bmatrix} + \begin{bmatrix}-3\\ -4\end{bmatrix}\right)$$

$$h_D^1 = ReLU\left(\begin{bmatrix}8.485\\ 4.243\end{bmatrix} + \begin{bmatrix}-3\\ -4\end{bmatrix}\right)$$

$$h_D^1 = ReLU\left(\begin{bmatrix}5.485\\ 0.243\end{bmatrix}\right)$$

$$h_D^1 = \begin{bmatrix}5.485\\ 0.243\end{bmatrix}$$

3 [7 points]. What is the second GCN layer output h_v^2 for node A and B?

Solution (3.)

$$h_A^2 = ReLU\left(W_2 * \left(\frac{[h_A^1]}{\sqrt{N(A)||N(v_A)}} + \frac{[h_B^1]}{\sqrt{N(B)||N(v_A)}}\right)\right)$$

$$h_A^2 = ReLU\left(\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix} *\right)$$

$$h_A^2 = ReLU\left(\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix} * \left(\frac{\begin{bmatrix}11.121\\ 4\end{bmatrix}}{\sqrt{1*1}} + \frac{\begin{bmatrix}13.864\\ 5.829\end{bmatrix}}{\sqrt{2*1}}\right)\right)$$

$$h_A^2 = ReLU\left(\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix} * \left(\frac{\begin{bmatrix}11.121\\ 4\end{bmatrix}}{1} + \frac{\begin{bmatrix}13.864\\ 5.829\end{bmatrix}}{\sqrt{2}}\right)\right)$$

$$h_A^2 = ReLU\left(\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix} * \left(\begin{bmatrix}11.121\\ 4\end{bmatrix} + \frac{1}{\sqrt{2}}\begin{bmatrix}13.864\\ 5.829\end{bmatrix}\right)\right)$$

$$h_A^2 = ReLU\left(\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix} * \begin{bmatrix}11.121\\ 4\end{bmatrix} + \frac{\sqrt{2}}{2}\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix} * \begin{bmatrix}13.864\\ 5.829\end{bmatrix}\right)$$

$$h_A^2 = ReLU\left(\begin{bmatrix}2 * 11.121 + 5 * 4\\ 4 * 11.121 + 2 * 4\end{bmatrix} + \frac{\sqrt{2}}{2}\begin{bmatrix}2 * 13.864 + 5 * 5.829\\ 4 * 13.864 + 2 * 5.829\end{bmatrix}\right)$$

$$h_A^2 = ReLU\left(\begin{bmatrix}22.242 + 20\\ 44.484 + 8\end{bmatrix} + \frac{\sqrt{2}}{2}\begin{bmatrix}27.728 + 29.145\\ 55.456 + 11.658\end{bmatrix}\right)$$

$$h_A^2 = ReLU\left(\begin{bmatrix}42.242\\ 52.484\end{bmatrix} + \frac{\sqrt{2}}{2}\begin{bmatrix}56.873\\ 67.114\end{bmatrix}\right)$$

$$h_A^2 = ReLU\left(\begin{bmatrix}42.242\\ 52.484\end{bmatrix} + \begin{bmatrix}40.215\\ 47.457\end{bmatrix}\right)$$

$$h_A^2 = ReLU\left(\begin{bmatrix}82.457\\ 99.941\end{bmatrix}\right)$$

$$h_A^2 = \begin{bmatrix}82.457\\ 99.941\end{bmatrix}$$

For Node B:

$$h_B^2 = ReLU\left(W_2 * \left(\frac{[h_A^1]}{\sqrt{N(A)||N(v_B)}} + \frac{[h_B^1]}{\sqrt{N(B)||N(v_B)}} + \frac{[h_C^1]}{\sqrt{N(C)||N(v_B)}}\right)\right)$$

$$h_B^2 = ReLU\left(W_2 * \left(\frac{[h_A^1]}{\sqrt{1*2}} + \frac{[h_B^1]}{\sqrt{2*2}} + \frac{[h_C^1]}{\sqrt{2*2}}\right)\right)$$

$$h_B^2 = ReLU\left(W_2 * \left(\frac{\begin{bmatrix}11.121\\ 4\end{bmatrix}}{\sqrt{2}} + \frac{\begin{bmatrix}13.864\\ 5.829\end{bmatrix}}{\sqrt{4}} + \frac{\begin{bmatrix}5.379\\ 0.172\end{bmatrix}}{\sqrt{4}}\right)\right)$$

$$h_B^2 = ReLU\left(\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix} * \left(\frac{\begin{bmatrix}11.121\\ 4\end{bmatrix}}{\sqrt{2}} + \frac{\begin{bmatrix}13.864\\ 5.829\end{bmatrix}}{\sqrt{4}} + \frac{\begin{bmatrix}5.379\\ 0.172\end{bmatrix}}{\sqrt{4}}\right)\right)$$

$$h_B^2 = ReLU\left(\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix} * \left(\frac{\begin{bmatrix}11.121\\ 4\end{bmatrix}}{\sqrt{2}} + \frac{\begin{bmatrix}13.864\\ 5.829\end{bmatrix}}{2} + \frac{\begin{bmatrix}5.379\\ 0.172\end{bmatrix}}{2}\right)\right)$$

$$h_B^2 = ReLU\left(\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix} * \left(\frac{1}{\sqrt{2}}\begin{bmatrix}11.121\\ 4\end{bmatrix} + \frac{1}{2}\begin{bmatrix}13.864\\ 5.829\end{bmatrix} + \frac{1}{2}\begin{bmatrix}5.379\\ 0.172\end{bmatrix}\right)\right)$$

$$h_B^2 = ReLU\left(\frac{\sqrt{2}}{2}\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix}\begin{bmatrix}11.121\\ 4\end{bmatrix} + \frac{1}{2}\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix}\begin{bmatrix}13.864\\ 5.829\end{bmatrix} + \frac{1}{2}\begin{bmatrix}2 & 5\\ 4 & 2\end{bmatrix}\begin{bmatrix}5.379\\ 0.172\end{bmatrix}\right)$$

$$h_B^2 = ReLU\left(\frac{\sqrt{2}}{2}\begin{bmatrix}2 * 11.121 + 5 * 4\\ 4 * 11.121 + 2 * 4\end{bmatrix} + \frac{1}{2}\begin{bmatrix}2 * 13.864 + 5 * 5.829\\ 4 * 13.864 + 2 * 5.829\end{bmatrix} + \frac{1}{2}\begin{bmatrix}2 * 5.379 + 5 * 0.172\\ 4 * 5.379 + 2 * 0.172\end{bmatrix}\right)$$

$$h_B^2 = ReLU\left(\begin{bmatrix}42.242\\ 52.484\end{bmatrix} + \begin{bmatrix}56.873\\ 21.86\end{bmatrix} + \begin{bmatrix}5.809\\ 10.93\end{bmatrix}\right)$$

$$h_B^2 = ReLU\left(\begin{bmatrix}64.115\\ 81.599\end{bmatrix}\right)$$

$$h_B^2 = \left[\begin{bmatrix}64.115\\ 81.599\end{bmatrix}\right]$$

4. [5 points] From the perspective of “message passing”, what does the k-th layer of GCN achieve in graph representation learning?

Solution (4).

In the k-th layer, each node aggregates the representations (messages) from its neighbors in the (k-1)-th layer, along with its own representation from the (k-1)-th layer. This aggregation is weighted by the learnable parameter matrix W_k and optionally normalized by the node degrees. The aggregated representation is then passed through a non-linear activation function (ReLU in this case) to obtain the node's new representation in the k-th layer. This message passing scheme allows the node representations to recursively incorporate and propagate information from their local neighborhoods, enabling the GCN to capture both node features and graph structure simultaneously in the learned representations. As the number of layers increases, nodes can integrate information from further away in the graph, learning higher-order representations that encode more global structural information.